

RFC-0057: API Rate Limiting

RFC: RFC-0057 · **Author:** Marta Silva (Senior Engineer, API Platform) · **Reviewers:** Hana Kowalczyk, Arjun Mehta · **Status:** Accepted on 8 April 2026 · **Shipped in:** Atlas 4.3

Summary

The Atlas public REST API currently has only a coarse global limit enforced in the load balancer. A handful of customers running aggressive polling scripts have caused two SEV2 incidents in 2026 by saturating the Query API. This RFC introduces per-API-key rate limits that depend on the customer's pricing tier, with standard response headers so that clients can back off gracefully.

Goals

- Protect shared infrastructure from noisy neighbours.
- Give each tier predictable, documented limits.
- Return consistent, machine-readable errors when limits are exceeded.

Non-goals

- Billing based on API usage.
- Limits on the Atlas web application (it uses an internal API with separate protections).

Algorithm

We use a **token bucket** per API key. Each bucket refills at the sustained rate of the tier and can hold at most the burst size. The bucket state is stored in the Redis cluster `redis-ratelimit`, and enforcement happens in the Envoy edge proxy through the global rate-limit service. If Redis is unavailable, Envoy fails open for up to 5 minutes and then falls back to a static limit of 50 requests per minute per key.

Limits per tier

REST API (management and query endpoints)

Tier	Sustained limit	Burst	Concurrent queries
Starter	100 requests per minute	20	2
Growth	1,000 requests per minute	200	10
Enterprise	10,000 requests per minute	2,000	50

Ingestion endpoint (`POST /v2/events`)

Tier	Events per second	Maximum batch size
Starter	500	1,000 events
Growth	5,000	1,000 events
Enterprise	50,000	1,000 events

Enterprise customers may request custom limits through their Technical Account Manager. Custom limits above 2x the standard Enterprise limit require approval from the API Platform team lead.

Response format

Every API response includes the headers:

- `X-RateLimit-Limit`: the sustained limit for the key
- `X-RateLimit-Remaining`: tokens left in the bucket
- `X-RateLimit-Reset`: seconds until the bucket is full again

When the bucket is empty the API returns HTTP **429 Too Many Requests** with error code **NW-E4290** and a `Retry-After` header in seconds. Ingestion requests that exceed the per-second limit also receive NW-E4290; they are never written to the dead-letter topic.

Rollout

Step	Description	Date
1	Headers only, no enforcement (observe mode)	21 April 2026
2	Enforcement for Starter workspaces	5 May 2026
3	Enforcement for all tiers, announced in Atlas 4.3 release notes	12 May 2026

During observe mode, 31 API keys exceeded their tier limit. Customer Success contacted the owners of those keys before enforcement started.

Monitoring and alerts

- Dashboard "API / Rate limiting" shows 429 responses per tier and the top 20 throttled keys.
- Alert when more than 5% of all API requests in 15 minutes return NW-E4290 (indicates misconfigured limits rather than abusive clients).
- Redis memory and eviction alerts for `redis-ratelimit` are covered by runbook RB-CACHE-02.

Alternatives considered

- **Fixed window counters.** Simpler, but allow double bursts at window boundaries. Rejected.
- **Limits in the application layer (FastAPI middleware).** Rejected because rejected requests would still consume application capacity.

Decision

Accepted on 8 April 2026. Owned by the API Platform team.